

Checklist:

The Competent Software Engineer

Competent Software Developer Checklist

Getting Things Done

Prioritize work

- ☐ Identify the single most important task or project each week.
- ☐ Confirm your priority with your manager or team when unsure.
- ☐ Deliver the top priority within the agreed timeframe.
- ☐ Say “no” or renegotiate when lower-priority work threatens the top priority.
- ☐ Communicate tradeoffs clearly when asked to take on extra work.

Unblock yourself

- ☐ Notice when you are blocked, rather than continuing without making progress.
- ☐ Use a rule of thumb: if there is no meaningful progress after 30–60 minutes, admit you are blocked.
- ☐ Try different approaches before asking for help:
 - ☐ Rubber-duck the problem.
 - ☐ Draw it on paper.
 - ☐ Read documentation and references.
 - ☐ Search online or internal resources.
 - ☐ Ask a Q&A forum or internal channel.
 - ☐ Use AI tools carefully and verify the answer.
 - ☐ Take a short break.
 - ☐ Start from scratch or undo recent changes.
- ☐ Ask teammates for help when needed.
- ☐ Explain what you tried before asking for help.

- ☐ Escalate politely when blocked by another person.
- ☐ Preserve relationships when escalating.

Break down work

- ☐ Split large projects into epics, stories, tasks, and subtasks.
- ☐ Break tasks down until each one is clear and actionable.
- ☐ Identify the “golden path” or smallest useful implementation.
- ☐ Separate edge cases into their own tasks.
- ☐ Prioritize work that gets you closer to something shippable.
- ☐ Add, remove, or revise tasks as you learn more.
- ☐ Avoid spending more time managing tasks than doing the work.

Estimate work

- ☐ Estimate using similar past work when possible.
- ☐ Ask teammates for examples when doing something new.
- ☐ Identify what is known, unknown, and risky.
- ☐ Prototype before estimating unfamiliar systems or technologies.
- ☐ Break work apart so each task contains fewer unknowns.
- ☐ Provide worst-case estimates when uncertainty is high.
- ☐ Update estimates when new information appears.

Seek mentors

- ☐ Identify people you can learn from.
- ☐ Look for more than one mentor:
 - ☐ Dedicated mentor.
 - ☐ Ad hoc mentor.
 - ☐ Internet mentor.
- ☐ Ask specific questions rather than vague ones.
- ☐ Learn from experienced engineers’ habits, decisions, and feedback.
- ☐ Verify advice from online sources or AI tools before using it.

Maintain goodwill

- ☐ Try reasonable self-service steps before asking others for help.
- ☐ Respect other people’s time.
- ☐ Come prepared when asking for help.
- ☐ Summarize the problem, what you tried, and your next step.
- ☐ Help others when they are stuck.

- ☐ Share useful knowledge with the team.
- ☐ Thank people who help you.
- ☐ Avoid working alone for long stretches when pairing or feedback would help.

Take initiative

- ☐ Finish expected work before taking on extra initiatives.
- ☐ Document unclear systems, decisions, or processes.
- ☐ Volunteer for investigations when appropriate.
- ☐ Research useful tools, frameworks, or internal systems.
- ☐ Talk with your manager about upcoming work.
- ☐ Offer help on smaller tasks when you have capacity.
- ☐ Avoid creating distractions by starting too many side efforts.

Coding

Practice coding

- ☐ Code regularly.
- ☐ Work on meaningful real-world problems.
- ☐ Ask for code reviews.
- ☐ Learn from repeated review comments.
- ☐ Pair with more experienced developers when possible.
- ☐ Build side projects when useful.
- ☐ Complete tutorials or structured training.
- ☐ Practice coding challenges or kata.
- ☐ Read and write code consistently.

Read code

- ☐ Review teammates' code even when you are not the main reviewer.
- ☐ Read code outside your immediate team or company when helpful.
- ☐ Study open-source projects that use similar languages or patterns.
- ☐ Understand what code does before changing it.
- ☐ Take notes on unfamiliar patterns, conventions, and architecture.
- ☐ Ask the original author questions when something is unclear.

Write readable code

- ☐ Use clear, self-explanatory names.
- ☐ Follow team naming and formatting conventions.
- ☐ Keep functions and classes small.
- ☐ Use well-structured classes and modules.
- ☐ Apply single responsibility.
- ☐ Avoid unnecessary complexity.
- ☐ Use DRY appropriately, without over-abstracting.
- ☐ Add comments where they explain “why,” not obvious “what.”
- ☐ Refactor continuously as code grows.
- ☐ Ask others whether the code is easy to understand.

Write quality code

- ☐ Use the right level of abstraction.
- ☐ Avoid excessive tiny classes or unnecessary layers.
- ☐ Hide implementation details when it reduces complexity.
- ☐ Handle errors consistently.
- ☐ Validate inputs, especially user inputs.
- ☐ Expect invalid responses from functions, APIs, or systems.
- ☐ Treat external inputs as potentially malicious.
- ☐ Expect exceptions and error states.
- ☐ Handle unknown API or system states explicitly.
- ☐ Prefer safe defaults when response states are unclear.

Software Development Skills

Become proficient in a language

- ☐ Learn the language fundamentals deeply.
- ☐ Understand syntax, types, data structures, and control flow.
- ☐ Learn standard error-handling approaches.
- ☐ Understand memory management and performance basics.
- ☐ Study how the language compiles or runs.
- ☐ Learn the main framework used with the language.
- ☐ Go deeper than “just enough to ship.”
- ☐ Learn a second language when useful.
- ☐ Compare language strengths and weaknesses.

Debug effectively

- ☐ Use debugger tools in your IDE.
- ☐ Set breakpoints and inspect variables.
- ☐ Step into, over, and out of functions.
- ☐ Inspect call stacks.
- ☐ Use conditional breakpoints or watchpoints when helpful.
- ☐ Learn from experienced developers while they debug.
- ☐ Use logging when a debugger is unavailable.
- ☐ Use paper debugging for complex logic.
- ☐ Write tests to reproduce bugs.
- ☐ Confirm the root cause before fixing.

Refactor regularly

- ☐ Refactor your own code after it works.
- ☐ Act on refactoring ideas from code reviews.
- ☐ Read through code to identify confusing or duplicated areas.
- ☐ Keep a prioritized list of refactoring opportunities.
- ☐ Use IDE refactoring tools.
- ☐ Start with small refactors.
- ☐ Test after each refactor.
- ☐ Refactor tests and production code.
- ☐ Avoid large risky refactors without a safety net.
- ☐ Make refactoring an everyday habit.

Test thoroughly

- ☐ Test code before requesting review or committing.
- ☐ Manually verify behavior when needed.
- ☐ Think through edge cases.
- ☐ Confirm stakeholder expectations for unusual cases.
- ☐ Write automated tests when practical.
- ☐ Learn unit, integration, screenshot, and end-to-end testing.
- ☐ Add regression tests for bugs.
- ☐ Treat testing as part of development, not a separate afterthought.

Tools of the Productive Developer

Master your local environment

- ☐ Learn your IDE or editor deeply.
- ☐ Set up shortcuts for frequent actions.
- ☐ Configure formatting and linting.
- ☐ Configure your color scheme and workflow for comfort.
- ☐ Learn search, replace, refactor, and navigation features.
- ☐ Learn how to run, debug, and test quickly.
- ☐ Use hot reload or fast feedback loops where available.
- ☐ Avoid slow edit-compile-run cycles when possible.
- ☐ Use playgrounds or scratch environments for quick experiments.

Learn frequently used tools

- ☐ Learn Git fundamentals:
 - ☐ Branching.
 - ☐ Rebasing.
 - ☐ Merging.
 - ☐ Conflict resolution.
 - ☐ Cherry-picking.
- ☐ Use a Git client or command line confidently.
- ☐ Learn command-line basics.
- ☐ Navigate files and directories from the terminal.
- ☐ Run scripts and tools from the terminal.
- ☐ Search files and set environment variables.
- ☐ Learn enough regular expressions for search and editing.
- ☐ Learn basic SQL.
- ☐ Understand SELECT, FROM, WHERE, ORDER BY, GROUP BY, HAVING, joins, and views.
- ☐ Use AI coding assistants carefully.
- ☐ Save useful prompts or workflows.
- ☐ Understand company-specific tools and systems.
- ☐ Create a personal productivity cheat sheet.

Iterate quickly

- ☐ Read existing code before changing it.
- ☐ Ask someone to walk you through unfamiliar systems.
- ☐ Draw classes, modules, and connections.
- ☐ Share your code map with teammates.

- ☐ Record useful “aha” moments in a cheat sheet.
- ☐ Understand the CI/CD pipeline.
- ☐ Know how to access CI/CD logs.
- ☐ Know how to access production logs, dashboards, and metrics.
- ☐ Understand safe processes for debugging production data.
- ☐ Make small pull requests when possible.
- ☐ Summarize pull requests clearly.
- ☐ Include screenshots or examples for UI changes.
- ☐ Mention covered and uncovered edge cases.
- ☐ Keep pull requests focused and easy to review.
- ☐ Request code reviews instead of waiting too long.
- ☐ Ask for feedback on your work.
- ☐ Compare your output and pace with team norms without treating it as a competition.

Quick Weekly Self-Review

- ☐ Did I deliver my most important work?
- ☐ Did I ask for help early enough when blocked?
- ☐ Did I break work into clear, shippable pieces?
- ☐ Did I communicate estimates, risks, and delays?
- ☐ Did I write readable, tested code?
- ☐ Did I learn something about the codebase, language, tools, or team?
- ☐ Did I get feedback from code review, pairing, users, or production?
- ☐ Did I help someone else or add goodwill to the team?
- ☐ Did I improve my workflow or tools?